SEGi University

BCL2113 Software Engineering

Applying Software Engineering Methodologies in LMS Development

Chan Jing Yi Student ID: SUOL2500321

Continuous Assessment: 40%

Proposed system: University Learning Management System

Requirements validation and real user elicitation remain pending.

Contents

1. Planning and analysis

2. Requirements engineering

3. Scrum process

4. System design

5. Team roles and project timeline

6. Software quality and testing

7. Risk management

8. Ethical and legal considerations

References

Appendix: Elicitation instruments

List of figures

Figure 1. LMS use case model

Figure 2. Assignment submission class model

List of tables

Table 1. Functional requirements

Table 2. Nonfunctional requirements

Table 3. Project timeline

Table 4. Assignment submission tests

Table 5. Risk register

1. Planning and analysis

A mid sized private university needs a learning management system (LMS) that brings course enrolment, learning materials, assignments, grades and communication into one accessible service. The proposed University LMS replaces a hypothetical fragmented arrangement in which staff distribute files through separate channels, students cannot reliably confirm a submission, and access to course material is difficult to control. These weaknesses are an illustrative baseline, not findings about a particular university. The proposed system provides a single course space with authorised access and a recorded assignment workflow.

The first objective is to provide students and lecturers with a consistent course workspace for enrolment, content access, communication and assignment management before the next academic year. The second is to protect and preserve academic information through explicit access controls, reliable submission receipts and measurable recovery procedures. These objectives prioritise the continuity of teaching as well as the confidentiality of records.

The design responds to known engineering concerns rather than claiming that every institution experiences the same failures. File uploads require authentication, validation and safe storage (OWASP, n.d.), while inaccessible interfaces can exclude students using assistive technology. The proposed accessibility target follows WCAG 2.2 (W3C, 2024).

2. Requirements engineering

2.1 Elicitation and evidence status

The brief requires elicitation from real users. No student or lecturer responses were supplied for this report, and no interviews or survey results are represented as having occurred. The specification below is a provisional proposal derived from the scenario and research, not validated user evidence. Before submission, conduct the interviews and survey in the appendix, record dates, roles and anonymised responses with consent, analyse themes, and revise the requirements. Without that evidence the elicitation and evidence based analysis criteria remain incomplete.

The proposed approach combines short interviews with students and lecturers and a concise survey for a wider group. Interviews uncover exceptions, such as unreliable connections near a deadline; the survey tests how widely needs recur. Include an administrator to discuss course provisioning and the handling of inaccessible accounts. Participation should be voluntary, and responses should avoid names or grades unless necessary. A facilitator records each theme and checks it with participants rather than treating a single opinion as a universal requirement.

The provisional analysis identifies five needs from the brief: controlled enrolment, reliable submissions, accessible course materials, lecturer assessment tools and course communication. For example, a submission can appear complete to a student while a file has not been stored. A server generated receipt and an authenticated submission history make completion observable. This is a design inference, not an interview finding.

2.2 User requirement specification

Actors are authenticated students, lecturers and administrators. A course membership record links a student to a course. A lecturer is assigned to a course by an administrator. Unless stated otherwise, only members and assigned lecturers can access that course. The following requirements are provisional and use identifiers for traceability.

Table 1. Functional requirements

ID	Requirement and acceptance condition
FR1	An authenticated student can request enrolment in an open course; the system records one membership per student and course and confirms the result.
FR2	A course lecturer can upload and publish course materials; enrolled students can view or download published items in that course.
FR3	An enrolled student can upload an allowed assignment file before the deadline. The system records server receipt time and returns a unique receipt after durable storage.
FR4	A course lecturer can create assignments, set a deadline and view submissions for that course.
FR5	A course lecturer can record and publish a grade and feedback; only the student and authorised staff can view the student's result.
FR6	Students and lecturers can post and read course announcements or messages according to their assigned permissions.
FR7	An administrator can create courses, assign lecturers and manage course membership.

Table 2. Nonfunctional requirements

ID	Requirement and verification
NFR1	Access control: automated tests shall reject access to another course or student's private submission and grade.
NFR2	Performance: at 500 concurrent active sessions, 95% of course page requests shall complete within 3 seconds, excluding file transfer, in a defined load test.
NFR3	Availability: target 99.5% monthly service availability excluding announced maintenance, measured by monitoring.
NFR4	Recovery: daily encrypted backups; demonstrate restoration to a test environment within 4 hours with at most 24 hours of data loss.
NFR5	Accessibility: relevant user journeys shall meet WCAG 2.2 AA success criteria, verified by automated and manual testing.
NFR6	Upload safety: allow PDF and DOCX files up to 20 MB; validate file type and scan before release, rejecting files that fail validation.

Requirements are analysed for conflicts and reviewed against user roles, exceptions and measurable outcomes. Each item receives an owner and acceptance test; the product owner maintains versions and decisions in a traceability matrix. NFR2 and NFR3 are proposed targets,

not measured results, and need stakeholder approval. Deadline policy is explicit: the server checks the time of completed receipt, and rejects a late submission unless a lecturer has recorded an extension. The team must validate these choices with actual users and university policy before baselining.

3. Software process model: Scrum

Scrum is suitable because students and lecturers can inspect working increments while the university refines priorities before launch. The product owner maintains and orders a product backlog, beginning with course access, submission receipts and privacy controls. The Scrum Master supports the process and removes impediments. Developers, including design and testing specialists, create a usable increment in each two week sprint (Schwaber and Sutherland, 2020).

During sprint planning the team chooses a sprint goal and backlog items that fit its capacity. Daily Scrum allows developers to inspect progress and adjust their plan. For example, a submission sprint could deliver upload validation, storage and the receipt as one vertical slice. During the sprint review, student and lecturer representatives inspect the increment and discuss backlog changes. The retrospective addresses how the team works, such as improving automated regression tests. The definition of done requires code review, passing tests, accessibility checks and an updated traceability record. Scrum supports adaptation, but does not itself guarantee delivery by a fixed date; the owner must manage scope, dependencies and release risks.

4. System design

The use case view identifies capabilities and actors within the system boundary. Each line indicates participation, not sequence. All access assumes authentication and course authorisation. FR identifiers connect the diagram to the specification.

Figure 1. LMS use case diagram

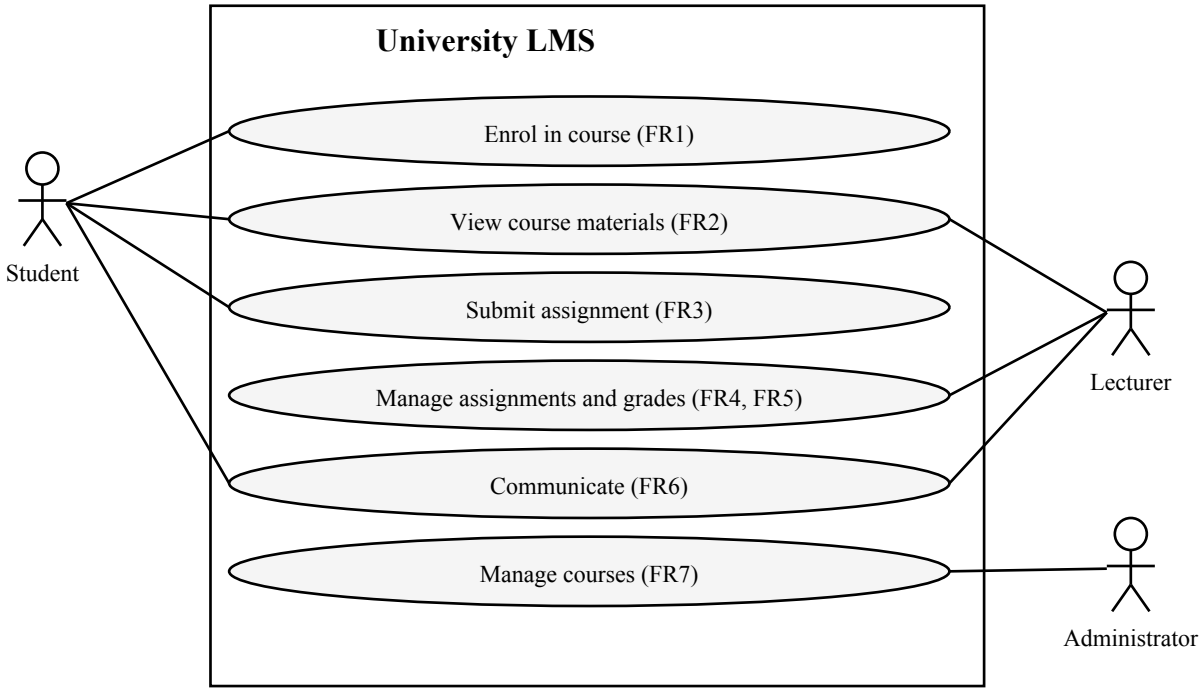
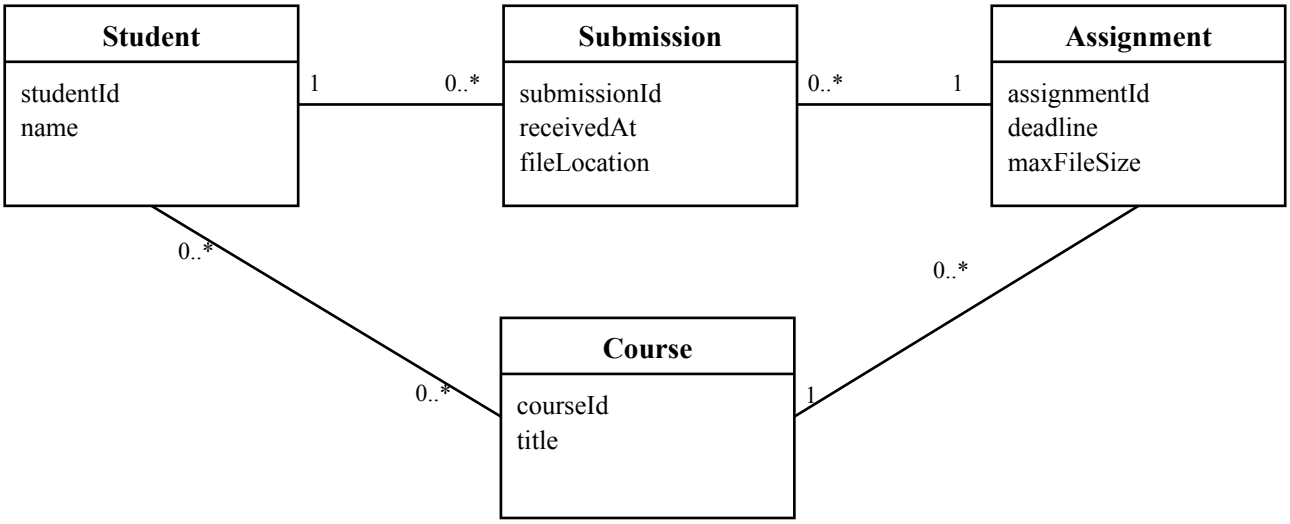


Figure 2. Assignment submission class diagram



A student can create many submissions; each submission belongs to one student and one assignment. An assignment belongs to one course, which may contain many assignments. A student can enrol in multiple courses and a course can have many students; an implementation should represent that association using a membership record. Submission stores a receipt time and a storage reference rather than embedding file contents in the class. Authorisation and deadline checks occur before the record is committed.

5. Team roles and project planning

The product owner represents university priorities, owns the backlog and accepts completed work. The Scrum Master facilitates Scrum and helps remove impediments. Developers design and implement the interface, application services and storage. A quality engineer designs acceptance tests, load tests and regression checks. A security and accessibility specialist reviews access control, uploads and inclusive interaction. Roles can be combined in a small team, but responsibility for approval and verification should remain explicit.

Table 3. Indicative timeline, assuming a 24 week delivery window

Weeks	Activity and deliverable
1 to 2	Stakeholder elicitation, baseline risks and initial backlog.
3 to 6	Course, identity and enrolment increments; sprint reviews.
7 to 12	Materials, assignments and submission receipts; reviews every two weeks.
13 to 16	Grades, feedback and messaging; integrate accessibility checks.
17 to 20	Security, performance and recovery tests; correct defects.
21 to 22	Pilot with volunteer users; prioritise and resolve launch blockers.
23 to 24	Training, migration rehearsal, final acceptance and release decision.

The schedule is a planning assumption, not a commitment that the start date or university calendar has been confirmed. Deployment is conditional on acceptance, risk review and a successful restore rehearsal. If time becomes constrained, secondary features should be deferred before security or submission integrity.

6. Software quality and testing plan

The selected feature is assignment submission. The tester prepares an enrolled student, another course user, an open assignment, an expired assignment and valid and invalid files. Unit tests check deadline comparison and file validation; integration tests check the application, storage and database transaction; system tests exercise the complete browser journey and permission boundaries. Acceptance tests ask a student and lecturer to verify the receipt and lecturer view. Load tests use the agreed workload and security tests include unauthorised and malicious uploads. No tests are claimed to have been executed.

Table 4. Sample assignment submission test cases

ID / type	Action and expected outcome
T1 / unit	At a time before the deadline, validate a PDF within the size limit. Expected: accepted.
T2 / unit	Use a completed receipt time after the deadline without an extension. Expected: rejected with an explanation.
T3 / integration	Store a valid file and commit its record. Expected: a unique receipt matches the saved record; a storage failure produces no successful receipt.
T4 / system	A nonmember attempts to submit to a course. Expected: access denied and no submission created.
T5 / system	Upload a disallowed or oversized file. Expected: rejected safely, with a clear message.
T6 / system	An enrolled student submits and the lecturer opens the record. Expected: receipt, timestamp and correct file are visible to authorised users only.
T7 / recovery	Restore a backup in an isolated test environment. Expected: integrity checks pass and recovery objectives are measured.

Entry criteria are a stable test environment, representative test data and approved acceptance criteria. Exit criteria are all critical tests passing, no unresolved critical security defects, and documented owner approval for remaining issues. Failures are logged with steps, evidence, severity, owner and retest status.

7. Risk management

The risk score is likelihood multiplied by impact, each on a scale from 1 to 5. Scores of 15 or more are high, 8 to 14 medium, and 1 to 7 low. The product owner reviews risks at each sprint review and assigns owners and triggers.

Table 5. Risk register

Risk	L × I / level	Mitigation and owner
Unauthorised access to grades	3 × 5 = 15 high	Security lead: least privilege, permission tests, monitoring and incident response.
Lost or incomplete submissions	3 × 5 = 15 high	Developer: durable storage, receipt after commit, backups and restoration drills.
Peak deadline traffic	4 × 4 = 16 high	Operations lead: load test, monitor capacity and establish an escalation plan.
Scope growth delays release	4 × 4 = 16 high	Product owner: order backlog, agree minimum release and review scope each sprint.
Inaccessible course journeys	3 × 4 = 12 medium	Accessibility specialist: test with assistive technology and users before acceptance.

8. Ethical and legal considerations

Privacy and security. The LMS processes personal information, including enrolment, submissions and grades. Malaysia's Personal Data Protection Act 2010 is relevant to the processing of personal data in commercial transactions; the university must assess its precise obligations, including applicable amendments and current guidance (Personal Data Protection Department, n.d.). The design should collect only necessary information, explain its use, restrict access by role, encrypt data in transit and backups, define retention periods and provide a process for correcting inaccurate records. Access logs should support accountability without unnecessary surveillance. Legal review is needed before deployment.

Accessibility and fairness. A student who cannot navigate an assignment form with a keyboard or screen reader may lose the opportunity to submit work. The team should target WCAG 2.2 AA, include keyboard and assistive technology checks, provide accessible feedback and involve disabled users in evaluation (W3C, 2024). Automated checks cannot establish accessibility alone. These measures address equitable participation; the institution should also check applicable accessibility obligations and provide reasonable support when problems arise.

References

OWASP (n.d.) File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Personal Data Protection Department (n.d.) Personal Data Protection Act 2010 (Act 709). https://www.pdp.gov.my/en/akta/pdp-act-2010-en/

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. https://scrumguides.org/scrum-guide.html

W3C (2024) Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

Appendix: Elicitation instruments

Proposed interviews: invite students, lecturers and an administrator. Ask what currently makes course access difficult; how they confirm an assignment has been received; what should happen if an upload fails near a deadline; who should see submissions and grades; what lecturers need to create and assess assignments; how participants use announcements or messages; and what accessibility or device constraints affect their work. Record consent, role, date and anonymised answers.

Proposed survey: ask participants to rate the importance of submission confirmation, clear deadlines, accessible materials, grade privacy and course communication on a five point scale. Ask how often they use a mobile device and invite one example of a failed or confusing course task. Summarise the number of respondents and response patterns without inventing findings. Map each supported need to a requirement identifier, document exceptions and obtain participant confirmation of the interpretation.

Evidence to complete: participant roles and count; interview or survey dates; anonymised responses; theme analysis; requirement changes; and stakeholder validation. These items are

pending actual participation.